

ForgeHTTP

Function Specification

A per-class breakdown of every function in the ForgeHTTP UML class diagram: what it does, what it returns, and what it should print where output matters.

Companion document to PLAN.md and the ForgeHTTP UML Class Diagram
Reference this alongside the diagram while implementing each milestone.

How to read this document

Each section below corresponds to one layer of the ForgeHTTP architecture (matching the color-coded clusters in the UML diagram). Within each layer, every class lists its constructor and public functions in the order they appear on the diagram.

For every function you will find: its signature, a description of what it must do, and — where the function produces visible output — an **Expected Output** block showing the exact formatting to match. Declare each class's interface in its `.h` file and implement the logic in the matching `.cpp` file, mirroring the include/src split in the repository structure.

Functions marked **[pure virtual]** must not be implemented in the declaring class — only overridden in subclasses. Functions marked **[static]** belong to the class itself, not an instance.

Memory ownership: wherever the UML shows a filled-diamond composition edge (e.g. Server owns Socket, ThreadPool owns Worker), the owning class is responsible for that object's lifetime — construct it in the owner's constructor (or on first use) and release it in the owner's destructor. Test every class for memory leaks before moving to the next milestone.

1 Networking Layer

Raw POSIX socket handling. No HTTP awareness lives here.

1.1 Socket

RAII wrapper around a POSIX file descriptor.

Declare in `Socket.h`, implement in `Socket.cpp`.

Socket()

Constructs a socket via `socket(AF_INET, SOCK_STREAM, 0)` and stores the resulting file descriptor. Throw if creation fails.

~Socket()

Closes the underlying file descriptor if it is still open. This is the class's core RAII guarantee — a `Socket` going out of scope must never leak a file descriptor.

void bind(int port)

Binds the socket to the given port on all interfaces. Sets `SO_REUSEADDR` before binding so the server can restart quickly without "address already in use" errors.

void listen(int backlog)

Marks the socket as passive and ready to accept incoming connections, with the given backlog queue size.

Socket accept()

Blocks until a client connects, then returns a new `Socket` wrapping the accepted connection's file descriptor. The listening socket itself keeps listening.

std::string receive()

Reads available bytes from the socket into a buffer and returns them as a string. Loops on `recv()` until the client's request has been fully read (or the connection closes).

void send(const std::string& data)

Writes the given bytes to the socket. Loops on `send()` to handle partial writes, since a single call is not guaranteed to send the whole buffer.

1.2 Connection

A thin, request-scoped handle around one accepted `Socket`.

Declare in `Connection.h`, implement in `Connection.cpp`.

Connection(Socket socket)

Takes ownership of an already-accepted `Socket` (composition — the `Connection` owns this `Socket` for its lifetime).

std::string read()

Delegates to the underlying `Socket`'s `receive()` to pull the raw request bytes off the wire.

void write(const std::string& data)

Delegates to the underlying `Socket`'s `send()` to write the raw response bytes back to the client.

void close()

Explicitly closes the connection ahead of destruction, e.g. after a response has been fully sent and keep-alive is not in use.

2 HTTP Layer

Turns raw bytes into structured request/response objects, and back again.

2.1 HttpMethod

Enum of supported HTTP verbs.

Declare in HttpMethod.h. No .cpp needed for a plain enum.

```
enum class HttpMethod { GET, POST, PUT, DELETE, PATCH }
```

Represents the verb of an HTTP request. Used by HttpRequest, Route, and the Router's registration functions so routes are matched on both path and method.

2.2 HttpRequest

A parsed, structured representation of an incoming HTTP request.

Declare in HttpRequest.h, implement in HttpRequest.cpp.

```
HttpMethod method / std::string path / std::string version
```

Populated by HttpParser from the request line, e.g. GET /users?id=10 HTTP/1.1 becomes method=GET, path=/users, version=HTTP/1.1.

```
std::map headers
```

All request headers, keyed by name (e.g. "Host", "Accept").

```
std::string body
```

The raw request body, if any (e.g. a JSON payload on a POST).

```
std::map query
```

Key/value pairs parsed from the query string portion of the path.

```
std::map params
```

Key/value pairs extracted from a matched route pattern, e.g. /users/:id matching /users/42 populates params["id"] = "42".

```
std::string getHeader(const std::string& key) const
```

Returns the value of the named header, or an empty string if it is not present. Lookup should be case-insensitive per the HTTP spec.

```
std::string getQueryParam(const std::string& key) const
```

Returns the value of the named query parameter, or an empty string if absent.

2.3 HttpResponse

A structured HTTP response, convertible to raw wire bytes.

Declare in HttpResponse.h, implement in HttpResponse.cpp.

```
int statusCode / headers / body
```

The response's status line code, header map, and body content.

```
static HttpResponse json(const std::string& data) [static]
```

Builds a 200 response with Content-Type: application/json and the given body, automatically computing Content-Length.

static HttpResponse text(const std::string& data) [static]

Builds a plain-text response with Content-Type: text/plain, again setting Content-Length automatically.

void setHeader(const std::string& key, const std::string& value)

Sets or overwrites a single response header.

std::string toString() const

Serializes the full response (status line, headers, blank line, body) into the exact bytes to write to the socket.

Expected Output:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 27

{"message":"Hello World"}
```

2.4 HttpParser

Stateless parser that turns raw request bytes into an HttpRequest.

Declare in HttpParser.h, implement in HttpParser.cpp.

static HttpRequest parse(const std::string& raw) [static]

Parses the request line, headers, and body from the raw socket bytes and returns a fully populated HttpRequest. Should tolerate missing trailing whitespace and both \r\n and \n line endings.

std::map parseHeaders(const std::string& raw)

Splits the header block into a map, one entry per Key: Value line.

std::map parseQueryString(const std::string& path)

Splits everything after ? in the path into key/value pairs, and returns the map (the caller strips the query string from the stored path).

3 Server Layer

Owns the full request lifecycle: listens, dispatches work, and shuts down cleanly.

3.1 Server

Top-level orchestrator. Knows about every other layer, but no layer below it knows about Server.

Declare in Server.h, implement in Server.cpp.

void start(int port)

Binds and listens on the given port (falling back to the PORT environment variable when no port is passed), then enters the accept loop. Prints a short startup banner.

Expected Output:

```
ForgeHTTP
-----
Server running on port 8080
Workers: 8
```

void stop()

Signals the accept loop to exit and shuts down the ThreadPool gracefully, letting in-flight requests finish before returning.

void acceptConnections() [private]

The main loop: blocks on the listening Socket's `accept()`, wraps each new connection, and enqueues it on the ThreadPool rather than handling it inline — this is what makes the server concurrent.

void handleConnection(Connection& conn) [private]

Runs on a worker thread. Reads the raw request, parses it via `HttpParser`, runs it through the middleware chain, finds a matching `Route` via the `Router`, invokes its handler, and writes the resulting `HttpResponse` back through the `Connection`. Publishes `RequestReceived/ResponseSent` events along the way.

4 Concurrency Layer

Fixed-size worker pool pulling tasks off a thread-safe queue, instead of spawning a thread per connection.

4.1 TaskQueue

Thread-safe queue of pending work, guarded by a mutex and condition variable.

Declare in TaskQueue.h, implement in TaskQueue.cpp.

void push(std::function task)

Locks the internal mutex, appends the task, then notifies one waiting worker via the condition variable.

std::function pop()

Blocks on the condition variable until a task is available (or the pool is shutting down), then removes and returns the front task.

bool empty() const

Returns whether the queue currently holds no pending tasks. Mainly useful for tests/metrics, not for the pop loop itself (which should block, not poll).

4.2 Worker

A single thread that repeatedly pulls from the shared TaskQueue and executes tasks.

Declare in Worker.h, implement in Worker.cpp.

Worker(int id, TaskQueue& queue)

Stores its id (for logging/debugging) and a reference to the shared queue. Does not start the thread itself — ThreadPool owns that.

void run()

The thread's main loop: repeatedly calls `queue.pop()` and executes the returned task, until told to stop.

4.3 ThreadPool

Owns a fixed set of Worker threads and the shared TaskQueue they pull from.

Declare in ThreadPool.h, implement in ThreadPool.cpp.

ThreadPool(int numThreads)

Constructs the TaskQueue, then constructs numThreads Worker objects, each backed by a real `std::thread` running `Worker::run()`.

void enqueue(std::function task)

Pushes a unit of work (typically "handle this one connection") onto the shared TaskQueue for the next available worker to pick up.

void shutdown()

Sets the stopping flag, wakes every worker via the condition variable, and joins all worker threads so the process can exit cleanly with no leaked threads.

5 Routing Layer

Maps an incoming method + path to the handler that should run.

5.1 Route

One registered endpoint: a method, a path pattern, and the handler to invoke on a match.

Declare in Route.h, implement in Route.cpp.

bool matches(const std::string& path) const

Returns whether the given concrete path matches this route's pattern, including patterns containing :param segments (e.g. /users/:id matches /users/42 but not /users or /users/42/edit).

std::map extractParams(const std::string& path) const

Given a path already confirmed to match, returns the named parameters pulled out of it, e.g. {"id": "42"}.

5.2 Router

Holds every registered Route and finds the one that matches an incoming request.

Declare in Router.h, implement in Router.cpp.

void get(const std::string& path, Handler handler)

Registers a GET route for the given path pattern.

void post(const std::string& path, Handler handler)

Registers a POST route for the given path pattern.

void put(const std::string& path, Handler handler)

Registers a PUT route for the given path pattern.

void del(const std::string& path, Handler handler)

Registers a DELETE route for the given path pattern. Named del rather than delete since the latter is a reserved keyword.

Route* match(const HttpRequest& request)

Scans the registered routes for one whose method and pattern match the request, returning a pointer to it (or nullptr if none match, which should produce a 404 response upstream).

6 Middleware Layer

Chain of Responsibility: each link inspects the request/response and decides whether to continue.

6.1 Middleware

Abstract base class for every middleware link in the chain.

Declare in `Middleware.h`. Do not implement `handle()` here — leave it pure virtual.

```
virtual void handle(HttpRequest& req, HttpResponse& res, std::function next) = 0  
[pure virtual]
```

The one method every middleware implements. Should perform its own work (logging, header injection, auth checks, etc.) and then call `next()` to continue down the chain — or omit the call to short-circuit the request (e.g. rejecting an unauthenticated request before it reaches the router).

```
Middleware* setNext(Middleware* m)
```

Links this middleware to the next one in the chain and returns it, enabling
`a.setNext(&b) -> setNext(&c);` -style chaining at server setup time.

6.2 LoggerMiddleware

Logs every request/response pair with method, path, status, and timing.

Declare in `LoggerMiddleware.h`, implement in `LoggerMiddleware.cpp`.

```
void handle(HttpRequest& req, HttpResponse& res, std::function next) override
```

Records a start timestamp, calls `next()` to let the rest of the chain (and the eventual controller) run, then prints one log line once `res` has been populated.

Expected Output:

```
[18:42:10] GET /api/users      200  12ms  
[18:42:15] GET /does-not-exist 404   2ms
```

6.3 CorsMiddleware

Adds permissive cross-origin headers so the React dashboard can call the API from a different origin during development.

Declare in `CorsMiddleware.h`, implement in `CorsMiddleware.cpp`.

```
void handle(HttpRequest& req, HttpResponse& res, std::function next) override
```

Sets `Access-Control-Allow-Origin` (and related headers) on the response, then calls `next()`.
Short-circuits immediately with a 204 for OPTIONS preflight requests.

6.4 AuthMiddleware

Rejects requests that don't carry valid credentials, before they reach the router.

Declare in `AuthMiddleware.h`, implement in `AuthMiddleware.cpp`.

```
void handle(HttpRequest& req, HttpResponse& res, std::function next) override
```

Checks for a valid `Authorization` header. If missing or invalid, sets a 401 response and returns without calling `next()`. Otherwise calls `next()` to continue the chain.

6.5 RateLimiterMiddleware

Caps how many requests a client can make in a given window.

Declare in RateLimiterMiddleware.h, implement in RateLimiterMiddleware.cpp.

void handle(HttpRequest& req, HttpResponse& res, std::function next) override

Tracks request counts per client (e.g. by IP), and if the caller has exceeded the configured threshold, sets a 429 response and returns without calling next ().

7 Controllers Layer

The actual application logic each route ultimately calls into.

7.1 HealthController

Trivial liveness endpoint used to confirm the server is up.

Declare in HealthController.h, implement in HealthController.cpp.

HttpResponse getHealth(const HttpRequest& req)

Returns a 200 JSON response confirming the server is running. Registered by the Server against GET /health.

Expected Output:

```
{
  "status": "ok"
}
```

7.2 UserController

Backs the example JSON API described in PLAN.md.

Declare in UserController.h, implement in UserController.cpp.

HttpResponse getUsers(const HttpRequest& req)

Returns a 200 JSON array of users. Registered against GET /api/users.

Expected Output:

```
[
  {"id": 1, "name": "Alice"},
  {"id": 2, "name": "Bob"}
]
```

HttpResponse createUser(const HttpRequest& req)

Parses the request body, validates it, adds a new user, and returns a 201 response with the created record (or a 400 response if the body is invalid). Registered against POST /api/users.

8 Observability Layer

Observer pattern: the server publishes lifecycle/request events; anything can subscribe without the server knowing who's listening. Cross-cutting — Server talks to this layer directly, rather than requests flowing through it.

8.1 ServerEvent

The data passed from Server to every subscriber on each publish() call.

Declare in ServerEvent.h. A plain data-holder — no .cpp needed unless you add helper methods.

```
enum class ServerEventType { RequestReceived, ResponseSent, ServerStarted,
ServerStopped }
```

Identifies what kind of event this is, so subscribers can decide what (if anything) to do with it.

```
ServerEventType type / std::chrono::system_clock::time_point timestamp
```

Every event carries its type and when it occurred.

```
std::string method / std::string path / int statusCode / double durationMs
```

Populated for request-level events (RequestReceived, ResponseSent) so MetricsCollector and Logger have something to aggregate or print. Left default/empty for lifecycle-only events (ServerStarted, ServerStopped).

8.2 EventObserver

Interface implemented by anything that wants to react to server events.

Declare in EventObserver.h as a pure abstract interface.

```
virtual void onEvent(const ServerEvent& event) = 0 [pure virtual]
```

Called by ServerEventPublisher whenever a subscribed-to event occurs (e.g. RequestReceived, ResponseSent, ServerStarted, ServerStopped).

8.3 ServerEventPublisher

Maintains the list of subscribers and notifies them when the Server publishes an event.

Declare in ServerEventPublisher.h, implement in ServerEventPublisher.cpp.

```
void subscribe(EventObserver* observer)
```

Adds an observer to the notification list (aggregation — the publisher does not own the observer's lifetime).

```
void publish(const ServerEvent& event)
```

Calls onEvent () on every subscribed observer, in subscription order.

8.4 Logger

An EventObserver that writes a human-readable line per event.

Declare in Logger.h, implement in Logger.cpp.

```
void onEvent(const ServerEvent& event) override
```

Formats and prints the event, e.g. a line noting the server started or stopped. (Request/response-level logging is handled by LoggerMiddleware; this Logger covers server lifecycle events.)

8.5 MetricsCollector

An EventObserver that aggregates counts and timings, exposed via /metrics.

Declare in MetricsCollector.h, implement in MetricsCollector.cpp.

void onEvent(const ServerEvent& event) override

Increments requestCount, updates the running average response time, and tracks error counts, based on the event received.

HttpResponse getMetrics()

Builds a JSON snapshot of the current counters. Registered against GET /metrics.

Expected Output:

```
{
  "requests": 12452,
  "active_connections": 17,
  "average_response_time_ms": 8.4,
  "errors": 31,
  "workers": 8
}
```

9 Implementation Checklist

Before moving from one milestone to the next in PLAN.md, confirm the relevant classes above satisfy the following:

- Every function signature declared in the .h matches what's implemented in the .cpp — no drift.
- Every class with a composition edge in the UML (Server→Socket, ThreadPool→Worker, Router→Route, Menu-style owned collections) frees what it owns in its destructor.
- Pure virtual functions (Middleware::handle, EventObserver::onEvent) are not implemented in their declaring base class.
- Output-producing functions match their Expected Output blocks exactly — including tabs, newlines, and punctuation — since these are the easiest things to get subtly wrong.
- No memory leaks: run under valgrind (or an equivalent sanitizer) after each milestone, not just at the end.
- Concurrency-sensitive classes (TaskQueue, ThreadPool, Worker) are exercised under real concurrent load, not just single-threaded tests.